

Member 6

Guest feedback

Gunathilake P.G.K.I. | IT25103600

Web-based Hotel Reservation System for Weddings
Serene Hotel · React -> Spring Boot -> Microsoft SQL Server

Your ownership

Major function: Guest feedback.
Supporting function: Rating summary and complaint status filtering.
Staff account: feedback.

How to describe the 75% milestone

This is a core-workflow evaluation milestone, not a measured claim against an official marking rubric. Nine concrete deliverables below are implemented; the three advanced work areas on page 6 remain. Shared authentication is an evaluation account setup, not production registration.

- Customer review and complaint form
- One-to-five rating validation
- Customer feedback list and detail view
- Customer edits while feedback is open
- Staff status and response updates
- Require a response before resolution
- Staff removal of inappropriate or duplicate entries
- Average rating and status filtering
- Ownership protection and activity history

1. Prepare and run locally

Install Git, Node.js 22 LTS, and JDK 17 or 21. The project launcher downloads Maven if it is not installed. Run commands in PowerShell from the indicated folder. First installation needs internet.

Fast rehearsal: explicitly select the demo profile

Use the complete project snapshot for a full-team rehearsal, or the shared foundation plus your member ZIP for a single-member rehearsal. Extract code into one project root. Keep frontend and backend beside each other.

```
cd backend
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=demo
```

Keep this terminal open. In a second terminal at the project root:

```
cd frontend
npm.cmd ci
npm.cmd run dev
```

Open <http://127.0.0.1:5173>. Confirm the banner says EVALUATION DEMO and the API is online. Demo uses a persistent H2 file under backend/data. It is an alternate rehearsal mode; the requested database remains SQL Server.

Evaluation sign-in

Choose feedback for your staff workspace. Use customer or customer2 to demonstrate customer isolation. All demo accounts use SereneDemo2026! Credentials are held only in browser memory; a page refresh signs you out.

Demo records

Each merged module seeds one illustrative record only when its table is empty in the demo profile. Dates are relative to startup. Prices, names and capacities are evaluation examples, not verified hotel inventory. Never use real customer data in these rehearsal accounts.

2. Connect Microsoft SQL Server

Install SQL Server Developer or Express and SQL Server Management Studio (SSMS). Use the running instance selected by your team. These instructions do not modify an existing server automatically.

- In SSMS, run database/create-database.sql to create SereneWeddings.
- Create a dedicated SQL login with a private password. Map it to SereneWeddings. For this development milestone grant db_ddladmin, db_datareader and db_datawriter.
- For SQL authentication, configure mixed-mode authentication if needed. In SQL Server Configuration Manager enable TCP/IP for your instance and confirm the actual TCP port. A named instance may not use 1433.
- Restart your instance only if its configuration changes require it. Verify its host and port with your team. Do not guess credentials.

In the backend PowerShell terminal, replace the placeholders locally:

```
$env:DB_URL="jdbc:sqlserver://localhost:1433;" +
"databaseName=SereneWeddings;encrypt=true;" +
"trustServerCertificate=true"
$env:DB_USER="serene_app"
$env:DB_PASSWORD="YOUR_LOCAL_PASSWORD"
$env:DEMO_PASSWORD="YOUR_EVALUATION_PASSWORD"
.\mvnw.cmd spring-boot:run
```

SQL Server is the default profile. The frontend banner should say SQL Server profile. Hibernate creates/updates tables during startup for this milestone; no demo records are seeded in this profile. Create your own test records through the UI.

Check persistence

Save a record, refresh its list, restart the backend, sign in again and verify the record remains. In SSMS inspect feedback_records. This environment did not have a running SQL Server service, so live SQL Server connectivity remains to be verified by your group.

Local development uses trustServerCertificate=true for local certificates. Before deployment use a trusted certificate, HTTPS, proper user management and versioned database migrations. The sample .env file documents variables; PowerShell does not load it automatically.

3. Commit only your member files

One of the six members acts as coordinator; there is no seventh member. First the coordinator commits 00-shared-foundation.zip to the team main branch and pushes it. Everyone creates their feature branch from that same foundation commit.

Use your member ZIP as a root-relative overlay. It contains only your feature UI, Java module, tests and member documents. It intentionally excludes package.json, lockfiles, shared CSS, security configuration and the root README.

```
git clone YOUR_TEAM_REPOSITORY_URL
cd serene-hotel-web
git switch main
git pull --ff-only origin main
git switch -c codex/member-6-feedback
```

Extract member-6-feedback.zip directly into this cloned repository root. The archive must create frontend/src/features/feedback, not an extra nested project folder. Review your files before committing.

```
git status --short
git add frontend/src/features/feedback
git add backend/src/main/java/lk/serene/feedback
git add backend/src/test/java/lk/serene/feedback
git add docs/members/member-6-feedback
git diff --cached --stat
git commit -m "Add member 6 feedback milestone"
git push -u origin codex/member-6-feedback
```

Open a pull request into main. The coordinator merges the six branches and runs the full checks. File sets in the six member archives are disjoint, so these supplied patches can merge in any order after the foundation.

Keep the boundary intact

Do not copy the complete snapshot onto a feature branch. Do not edit shared files independently. If a shared change is needed, ask the coordinator to apply it once, then pull it into each branch. Independent later edits can still create conflicts; the ZIP layout cannot prevent every future conflict.

4. Demonstrate your function

Rehearse this sequence before the evaluation. Use made-up names and addresses. Show both a successful action and a rejected action; explain the server rule behind the result.

1. Sign in as customer. Create a COMPLAINT with a booking reference, customer name, rating 2, and a clear complaint message.
2. Edit the message while the status is OPEN. Notice that customers do not receive staff resolution controls.
3. Sign in as feedback and update the complaint to IN_PROGRESS. Show search, status filters and average rating.
4. Attempt to mark it RESOLVED without a response; show the validation error. Add an explanation and save RESOLVED.
5. Sign in as customer and open View / edit to see the response. Staff may remove inappropriate or duplicate feedback; every removal leaves an audit entry.

Files to open when explaining the code

frontend/src/features/feedback/index.jsx defines your labels, form fields and module metadata. Custom components, where present, stay in the same feature folder.

backend/src/main/java/lk/serene/feedback/Feedback.java defines the stored fields and validation.

FeedbackRepository.java provides persistence.

FeedbackController.java applies your module business rules.

backend/src/test/java/lk/serene/feedback/FeedbackTest.java proves the main rules.

Follow a request

React Workbench -> api.js -> /api/feedback -> Spring Security -> shared CrudController ->

FeedbackController.validate -> JPA repository -> database. The response updates the table. For errors, the UI displays the API message instead of pretending the save succeeded.

5. Verify, troubleshoot, continue

Run your tests and the frontend build

```
cd backend
.\mvnw.cmd "-Dtest=FeedbackTest" test
# Full integrated checks:
.\mvnw.cmd test
cd ../frontend
npm.cmd run build
```

The prepared full project passed 38 backend integration tests and a React production build. Automated backend tests use an isolated H2 database in SQL Server compatibility mode. This is not a substitute for running the group's actual SQL Server instance.

Common problems

- 401: wrong account/password or backend restarted with a different DEMO_PASSWORD. Sign out and sign in again.
- 403: wrong department, another customer's record, or a staff-only action. Use your assigned account.
- 409: refresh after another edit or a booking conflict. Do not silently overwrite.
- API OFFLINE: start Spring Boot and inspect its terminal. Verify ports 8080 and 5173. If Vite chooses another port, open the URL it prints.
- Maven cannot start: confirm JAVA_HOME points to a JDK with javac.exe. The first launcher run needs access to Maven Central.

Remaining advanced work for your function

Verified attendance checks, public review moderation, and automated complaint escalation.

Shared limitations to state honestly

Predefined evaluation users; no registration, password reset or profile management. Cross-module booking references are manual. No live payment gateway. No booking emails. MailerLite requires your credentials and a live test. No production deployment or SQL Server runtime verification was performed here.

Source: supplied proposal 2026-Y2-S1-MLB-B6G2-09.pdf, sections 6-7, plus the user's current scope. Member numbers follow function order, not the proposal cover-page order. Technical reference: developers.mailerlite.com/api/subscribers (Member 4 integration).